IMPLEMENTAREA CONCURENTEI IN LIMBAJE DE PROGRAMARE

CONCURENTA IN JAVA

Ioana Leustean

- Colectii concurente: asigura concurenta si sincronizare, fara a fi blocate de un singur lacat

- **BlockingQueue** - folosite in special pentru modelul **producator-consumator**
- **ConcurrentHashMap –** permite citiri concurente dar si un numar de scrieri concurente

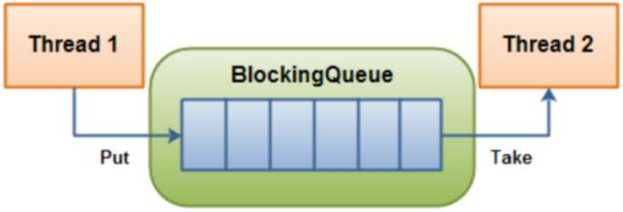
- Sincronizarea thread-urilor

- **CountDownLatch**: sincronizarea thread-urilor care au executat anumite task-uri
- **CyclicBarrier**: sincronizarea thread-urilor care asteapta la bariera

java.util.concurrent (Java SE 23 & JDK 23)

➢ Colectii concurente: **BlockingQueue**

```
public interface BlockingQueue<E>
extends Queue<E>
```



A BlockingQueue with one thread putting into it, and another thread taking from it.
http://tutorials.jenkov.com/java-util-concurrent/blockingqueue.html

Coada cu operatii care:

- blocheaza thread-urile care insereaza elemente cand coada este plina
- blocheaza thread-urile care scot elemente cand coada este goala

Constructori

- **ArrayBlockingQueue(int capacity)**
- **LinkedBlockingQueue() / LinkedBlockingQueue(int capacity)**

Metode

Summary of BlockingQueue methods

	Throws exception	*Special value*	*Blocks*	*Times out*
Insert	`add(e)`	`offer(e)`	`put(e)`	`offer(e, time, unit)`
Remove	`remove()`	`poll()`	`take()`	`poll(time, unit)`
Examine	`element()`	`peek()`	[illegible]	[illegible]*licable*

put() si **take()** sunt operatii care blocheaza

BlockingQueue (Java SE 23 & JDK 23)

Exemplu: Producator-Consumator folosind **BlockingQueue**

```
public static void main(String[] args) {
   BlockingQueue<String> drop= new LinkedBlockingQueue<String> ();

      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCConsumer(drop))).start();
   }
```

```
class PCConsumer implements Runnable {
   private BlockingQueue<String> drop;
   public PCConsumer(BlockingQueue<String> drop) { this.drop = drop;}

   public void run() {
     while (true) {
       try {
         System.out.format("Message received: %s%n", drop.take());
         }
         catch (InterruptedException ex) { }     }}}
```

http://javarevisited.blogspot.ro/2012/02/producer-consumer-design-pattern-with.html

```
class PCProducer implements Runnable {
   private BlockingQueue<String> drop;

   public PCProducer(BlockingQueue<String> drop) {this.drop = drop;}

   public void run() {
      String importantInfo[] = { "m1", "m2", "m3", "m4" };
      Random random = new Random();

      for (int i = 0; i < importantInfo.length; ++i) {
         try {
         drop.put(importantInfo[i]);
         Thread.sleep(random.nextInt(1000));
         }
         catch (InterruptedException ex) { }
      }}}
```

```
Message received: m1
Message received: m1
Message received: m1
Message received: m2
Message received: m3
Message received: m2
Message received: m4
Message received: m2
Message received: m3
Message received: m3
Message received: m4
Message received: m4
```

```java
class PCProducer implements Runnable {
   private BlockingQueue<String> drop;

   public PCProducer(BlockingQueue<String> drop) {this.drop = drop;}

   public void run() {
      String importantInfo[] = { "m1", "m2", "m3", "m4" };
      Random random = new Random();

      for (int i = 0; i < importantInfo.length; ++i) {
         try {
         drop.put(importantInfo[i]);
         System.out.format("Message sent: %s%n", importantInfo[i]);
         Thread.sleep(random.nextInt(1000));
         }
         catch (InterruptedException ex) { }
      }}}
```

```
Message sent: m1
Message sent: m1
Message sent: m1
Message received: m1
Message received: m1
Message received: m1
Message sent: m2
Message received: m2
Message sent: m2
Message received: m2
Message sent: m3
Message received: m3
Message sent: m2
Message received: m2
Message sent: m3
Message received: m3
Message sent: m4
Message received: m4
Message sent: m4
Message received: m4
Message sent: m3
Message received: m3
Message sent: m4
Message received: m4
```

Exemplu: Producator-Consumator folosind **SynchronousQueue**

```
public static void main(String[] args) {
   BlockingQueue<String> drop= new SynchronousQueue<String> ();

      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCConsumer(drop))).start();
   }
```

- Producatorul si consumatorul sunt thread-uri sincronizate, orice operatie se face numai in prezenta unui thread care face **put()** si a unuia care face **take()**

- Elementele sunt transferate de la producator la consummator, fara a fi tinute in coada.

- Coada nu are capacitate (nici macar 1), este de fapt un punct de intalnire (rendezvous point, handoff)

[SynchronousQueue (Java SE 23 & JDK 23)]

```
Message sent:  m1
Message received: m1
Message received: m1
Message sent:  m1
Message received: m1
Message sent:  m1
Message sent:  m2
Message received: m2
Message sent:  m2
Message received: m2
Message sent:  m3
Message received: m3
Message sent:  m2
Message received: m2
Message sent:  m3
Message received: m3
Message sent:  m4
Message received: m4
Message sent:  m4
Message received: m4
Message sent:  m3
Message received: m3
Message sent:  m4
Message received: m4
```

- **LinkedBlockingQueue / SynchronousQueue**

```
public static void main(String[] args) {
   BlockingQueue<String> drop= new LinkedBlockingQueue<String> ();

      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
 //     (new Thread(new PCConsumer(drop))).start();
   }
```

Thread-urile producator executa **put**

```
Message sent: m1
Message sent: m1
Message sent: m1
Message sent: m2
Message sent: m3
Message sent: m4
Message sent: m2
Message sent: m2
Message sent: m3
Message sent: m3
Message sent: m4
Message sent: m4
```

```
public static void main(String[] args) {
   BlockingQueue<String> drop= new SynchronousQueue <String> ();

      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
      (new Thread(new PCProducer(drop))).start();
 //     (new Thread(new PCConsumer(drop))).start();
   }
```

Thread-urile producator sunt blocate, deoarece nu exista consumatori

```
PS C:\Users\igleu\Documents\DIR\ICLP\ICLP-2025-2026\pgJava\pg4> java ProducerConsumerSQ.java
```

http://javarevisited.blogspot.ro/2012/02/producer-consumer-design-pattern-with.html

- **CountDownLatch** sincronizarea thread-urilor care au executat anumite task-uri

- la creare se transimte numarul de taskuri: **latch =new CountDownLatch(n)**
- fiecare thread care executa un task decrementeaza **n** apeland **latch.countDown()**;
- thread-urile care apeleaza **latch.await()** sunt blocate pana cand **n=0**

```
class TaskE implements Runnable {
    private  AtomicInteger counter;
    private  CountDownLatch controller;
    TaskE (CountDownLatch c, AtomicInteger counter){this.controller = c; this.counter =counter;}

public void run () {
        for (int i = 0; i < 100; i++) {counter.incrementAndGet();}

        controller.countDown();

        for (int i = 0; i < 1000; i++) { counter.incrementAndGet();} }
```

Task-ul executat de fiecare thread Incrementeaza un contor si semnaleza prin controller cand a incrementat de 100 de ori

- **CountDownLatch** sincronizarea thread-urilor care au executat anumite task-uri

- la creare se transimte numarul de taskuri: **latch =new CountDownLatch(n)**
- fiecare thread care executa un task decrementeaza **n** apeland **latch.countDown()**;
- thread-urile care apeleaza **latch.await()** sunt blocate pana cand **n=0**

```
public static void main (String[] args) throws InterruptedException  {
      CountDownLatch controller = new CountDownLatch(3);

     ExecutorService pool = Executors.newCachedThreadPool();
      AtomicInteger counter = new AtomicInteger(0);

     for(int i=0;i<3;i++) {pool.execute(new TaskE(controller,counter));};

      controller.await();

      System.out.println(counter);
      pool.shutdown();            // try {if  (!pool.awaitTermination() ){pool.shutdownNow} catch{}
      System.out.println(counter);}
```

n=3

in acest exemplu dorim sa stim cand fiecare a incrementat contorul de 100 de ori

➢ CountDownLatch

```
class TaskE implements Runnable {
    private  AtomicInteger counter;
    private  CountDownLatch controller;
    TaskE (CountDownLatch c, AtomicInteger counter){this.controller = c; this.counter =counter;}

public void run () {
         for (int i = 0; i < 100; i++) {counter.incrementAndGet();}
        controller.countDown();
        for (int i = 0; i < 1000; i++) { counter.incrementAndGet();} }
```

```
public static void main (String[] args) throws InterruptedException  {
      CountDownLatch controller = new CountDownLatch(3);
     ExecutorService pool = Executors.newCachedThreadPool();
      AtomicInteger counter = new AtomicInteger(0);
     for(int i=0;i<3;i++) {pool.execute(new TaskE(controller,counter));};
      controller.await();
      System.out.println(counter);
      pool.shutdown();
      System.out.println(counter);}
```

afiseaza o valoare **>= 300**
task-urile sunt lasate sa isi termine executia
afiseaza **3300**

```
PS C:\Users\
1401
3300
PS C:\Users\
3166
3300
PS C:\Users\
PS C:\Users\
2386
3300
PS C:\Users\
1736
3300
```

➢ **CyclicBarrier** sincronizarea thread-urilor care asteapta la bariera

- la creare se transmite numarul de thread-uri si, optional, o actiune (instanta a lui Runnable), care se executa cand toate thread-urile au ajuns la bariera
- thread-urile care apeleaza **barrier.await()** poate trece mai departe numai dupa ce toate au ajuns la bariera

```
class TaskE implements Runnable {
 private  AtomicInteger counter;
 private  CyclicBarrier barrier;
 TaskE (CyclicBarrier b, AtomicInteger counter){this.barrier = b; this.counter =counter;}

public void run () {
        for (int i = 0; i < 100; i++) {
            counter.incrementAndGet();  };

        try {barrier.await();} catch (Exception e) {
        System.out.println(e);return; }

       for (int i = 0; i < 1000; i++) {
            counter.incrementAndGet();}
        System.out.println(Thread.currentThread().getName()+": "+counter); }
```

Task-ul executat de fiecare thread incrementeaza un contor si asteapta la bariera pana fiecare thread a incrementat contorul de 100 de ori

➢ **CyclicBarrier** sincronizarea thread-urilor care asteapta la bariera

- la creare se transmite numarul de thread-uri si, optional, o actiune (instanta a lui Runnable), care se executa cand toate thread-urile au ajuns la bariera
- thread-urile care apeleaza **barrier.await()** poate trece mai departe numai dupa ce toate au ajuns la bariera

```
public static void main (String[] args) throws InterruptedException  {
   CyclicBarrier barrier = new CyclicBarrier(4, ()->{System.out.println("Threads reached thebarrier:"+counter);});

    ExecutorService pool = Executors.newCachedThreadPool();
    AtomicInteger counter = new AtomicInteger(0);

    for(int i=0;i<3;i++) {pool.execute(new TaskE(controller,counter));};

     controller.await();

     pool.shutdown();
  // try {if  (!pool.awaitTermination() ){pool.shutdownNow} catch{}
    System.out.println("Main thread: " + counter);}
```

Mesajul va fi afisat dupa ce toate thread-urile au ajuns la bariera.
Valoarea contorului ar trebui sa fie 300

n=4 am numarat si thread-ul principal

➢ **CyclicBarrier** sincronizarea thread-urilor care asteapta la bariera

- la creare se transmite numarul de thread-uri si, optional, o actiune (instanta a lui Runnable), care se executa cand toate thread-urile au ajuns la bariera
- thread-urile care apeleaza **barrier.await()** poate trece mai departe numai dupa ce toate au ajuns la bariera

```
public static void main (String[] args) throws InterruptedException  {
    CyclicBarrier barrier = new CyclicBarrier(4, ()->{System.out.println("Threads reached thebarrier:"+counter);});

    ExecutorService pool = Executors.newCachedThreadPool();
    AtomicInteger counter = new AtomicInteger(0);

     for(int i=0;i<3;i++) {pool.execute(new TaskE(controller,counter));};

      controller.await();

      pool.shutdown();
   // try {if  (!pool.awaitTermination() ){pool.shutdownNow} catc
    System.out.println("Main thread: " + counter);}
```

Mesajul va fi afisat dupa ce toate thread-urile au ajuns la bariera.
Valoarea contorului ar trebui sa fie 300

```
Threads reached the barrier: 300
pool-1-thread-3: 3300
pool-1-thread-1: 3134
pool-1-thread-2: 3098
Main thread: 3300
```

➢ **CyclicBarrier** pentru Task-uri si **CountDownLatch** pentru terminarea thread-urilor

```
class TaskE implements Runnable {
 private  AtomicInteger counter;   private  CyclicBarrier barrier; private  CountDownLatch controller;
 TaskE (CountDownLatch c, CyclicBarrier b, AtomicInteger counter){this.controller=c;this.barrier = b; this.counter=counter;}

public void run () {
        for (int i = 0; i < 100; i++) {counter.incrementAndGet(); };
        try {barrier.await();} catch (Exception e) {System.out.println(e);return;}
        for (int i = 0; i < 1000; i++) {counter.incrementAndGet();};
        System.out.println(Thread.currentThread().getName()+": "+counter);
        controller.countDown(); }
```

```
public static void main (String[] args) throws InterruptedException  {
      CountDownLatch controller = new CountDownLatch(3);
      CyclicBarrier barrier = new CyclicBarrier(4, ()->{System.out.println("Threads reached thebarrier:"+counter);});
     ExecutorService pool = Executors.newCachedThreadPool();
      AtomicInteger counter = new AtomicInteger(0);
     for(int i=0;i<3;i++) {pool.execute(new TaskE(controller, barrier, counter));};
      barrier.await(); controller.await();
     pool.shutdown();
     System.out.println("All done! Final value: " + counter); }
```

➢ **CyclicBarrier** pentru Task-uri si **CountDownLatch** pentru terminarea thread-urilor

```
class TaskE implements Runnable {
 private  AtomicInteger counter;   private  CyclicBarrier barrier; private  CountDownLatch controller;
 TaskE (CountDownLatch c, CyclicBarrier b, AtomicInteger counter){this.controller=c;this.barrier = b; this.counter=counter;}

public void run () {
        for (int i = 0; i < 100; i++) {counter.incrementAndGet(); };
        try {barrier.await();} catch (Exception e) {System.out.println(e);retu
        for (int i = 0; i < 1000; i++) {counter.incrementAndGet();};
        System.out.println(Thread.currentThread().getName()+": "+counter
        controller.countDown(); }
```

```
public static void main (String[] args) throws InterruptedException  {
      CountDownLatch controller = new CountDownLatch(3);
      CyclicBarrier barrier = new CyclicBarrier(4, ()->{System.out.println("Th
     ExecutorService pool = Executors.newCachedThreadPool();
      AtomicInteger counter = new AtomicInteger(0);
     for(int i=0;i<3;i++) {pool.execute(new TaskE(controller, barrier, counter)
      barrier.await(); controller.await();
     pool.shutdown();
     System.out.println("All done! Final value: " + counter); }
```

```
PS C:\Users\igleu\Documents\DIR\ICLP\
Threads reached the barrier: 300
pool-1-thread-3: 1323
pool-1-thread-1: 3300
pool-1-thread-2: 3053
All done! Final value: 3300
PS C:\Users\igleu\Documents\DIR\ICLP\
Threads reached the barrier: 300
pool-1-thread-3: 2853
pool-1-thread-1: 3300
pool-1-thread-2: 1300
All done! Final value: 3300
PS C:\Users\igleu\Documents\DIR\ICLP\
```

- **public class ConcurrentHashMap<K,V>**

- colectia este impartita in fragmente care sunt prelucrate in parallel
- colectia poate fi create cu un anume nivel de concurenta; numarul de fragmente prelucrate in paralel este dat de nivelul de concurenta
- cand un thread executa o operatie care blocheaza, este blocat numai fragmentul corespunzator
- actualizarile sunt operatii care blocheaza, regasirile nu blocheaza; este regasita ultima valoare modificata de o actualizare care s-a finalizat
- actiunile dintr-un thread care plaseaza un obiect in colectie sunt in relatie happens-before cu actiunile dintr-un alt thread care urmeaza accesarii/stergerii elementului din colectie
- colectia nu este ordonata, elementele pot fi procesate in paralel in ordini diferite
- piscina de thread-uri este creata cu ForkJoinPool.commonPool()

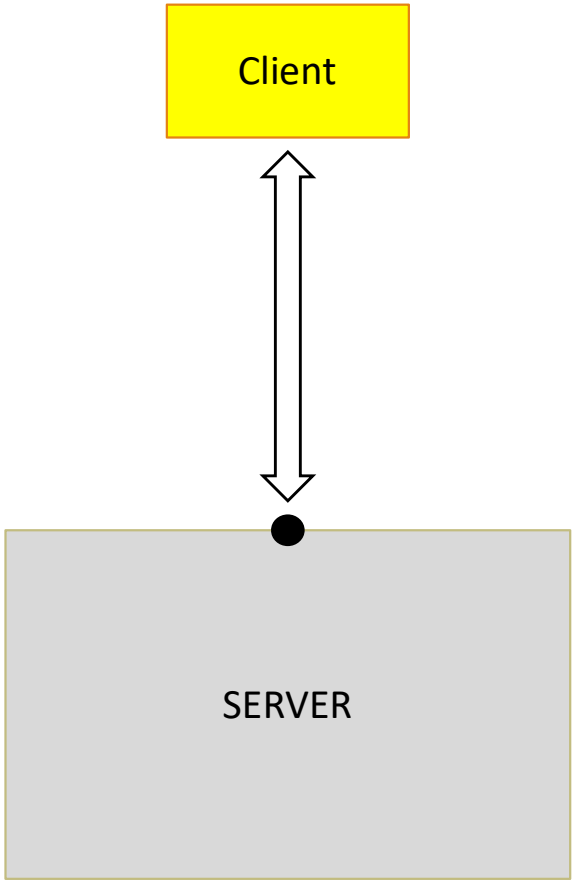
ConcurrentHashMap (Java SE 23 & JDK 23)

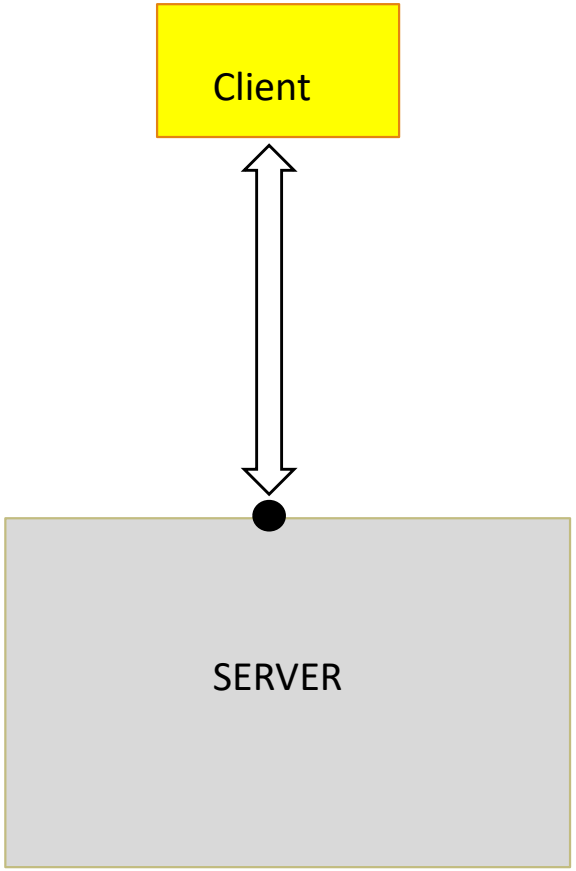
```
public class ConcHM{

private static volatile boolean ok=true;

public static void main(String[] args) throws Exception {
        ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<>();
        CountDownLatch controller = new CountDownLatch(3);
        (new WriterThread(map, "W1",controller)).start();
        (new WriterThread(map, "W2",controller)).start();
        (new ReaderThread(map, "R1")).start();
        (new ReaderThread(map, "R2")).start();
        (new ReaderThread(map, "R3")).start();
        controller.await();
        ok=false;
        }
private static class WriterThread extends Thread {…}
private static class ReaderThread extends Thread {…}

}
```

```
private static class ReaderThread extends Thread {
	private ConcurrentHashMap<Integer, String> map;
	private String name;
	public ReaderThread(ConcurrentHashMap<Integer, String> map, String threadName) {
	this.map= map; this.name = threadName;}
	public void run() {
	while (ok) {
	// long time = System.currentTimeMillis();
	String output = name + ": ";
	for (Integer key : map.keySet()) {
	String value = map.get(key);
	output += "("+key + "," + value + ") ";
	}
	System.out.println(output);
	try { Thread.sleep(ThreadLocalRandom.current().nextInt(300));} catch (InterruptedException ex) {}
	}}
	}
```

```
private static class WriterThread extends Thread {
   private ConcurrentMap<Integer, String> map;
   private CountDownLatch controller;
   private String name;
   public WriterThread(ConcurrentMap<Integer, String> map, String threadName, CountDownLatch c) {
   this.map= map; this.name = threadName; this.controller=c;}
   public void run() {
   String value = name;
   while (ok) {
   for (int i = 0; i < 3; i++) {
   Integer key = ThreadLocalRandom.current().nextInt(100);
 // long time = System.currentTimeMillis();
   if(map.putIfAbsent(key, value) == null) {
   System.out.println(name+" has put ("+key+","+value+")") ;}
   else {
      System.out.println(name+" duplicate") ;}}
      controller.countDown();
   try { Thread.sleep(ThreadLocalRandom.current().nextInt(500));} catch (InterruptedException ex) {}

   }}}
```

```
W2 has put (85,W2)
R1: (52,W1) (85,W2)
W1 has put (52,W1)
R3: (52,W1) (85,W2)
R2: (52,W1) (85,W2)
W2 has put (25,W2)
W1 has put (38,W1)
W2 has put (4,W2)
W1 has put (81,W1)
R2: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
R2: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
R3: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
R1: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
R1: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
R3: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
R2: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
R1: (81,W1) (52,W1) (4,W2) (85,W2) (38,W1) (25,W2)
W2 has put (56,W2)
W2 has put (74,W2)
W2 has put (16,W2)
```


Client
SERVER



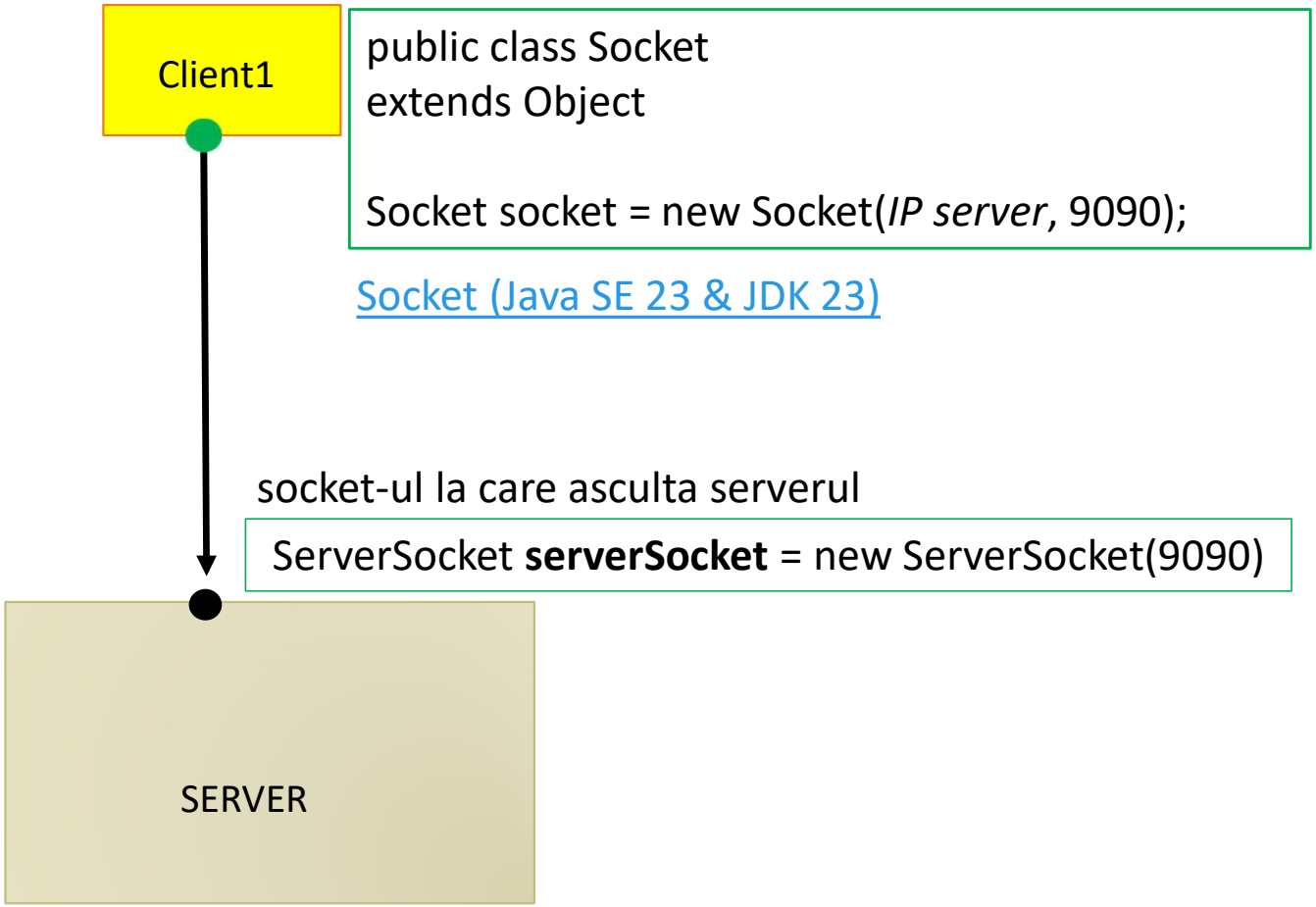
- un socket este un punct final in comunicarea bidirectionala dintre doua programe din aceeasi retea
- un socket are asociat un port

● un socket de server asteapta cererile venite din retea

```
public class ServerSocket
    extends Object

ServerSocket serverSocket = new ServerSocket(9090)
```

ServerSocket (Java SE 23 & JDK 23)

Clientul initiaza conexiunea creand un socket



Clientul initiaza conexiunea creand un socket

Client2

Client 1

Socket-ul serverului asteapta alte solicitari

Socket socket = new Socket(*IP server*, 9090);

socket-ul la care asculta serverul

ServerSocket **serverSocket** = new ServerSocket(9090)

Serverul accepta conexiunea creand un socket dedicat clientului

Socket **clientSocket = serverSocket.accept();**

SERVER

https://docs.oracle.com/javase/tutorial/essential/concurrency

```
public class Server {
   public static void main(String args[])
   {  ServerSocket serverSocket = new ServerSocket(9090);
      Socket clientSocket = serverSocket.accept();
                                                // comunicarea cu clientul
      BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
… }
```

prin **in** si **out** stabilesc canalele de comunicare

```
public class Client {
   public static void main(String args[])
   { Socket socket = new Socket("localhost", 9090);
                                                // comunicarea cu serverul
     PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
     BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
…}
```

https://www.geeksforgeeks.org/how-to-create-a-simple-tcp-client-server-connection-in-java/
Writing the Server Side of a Socket (The Java™ Tutorials > Custom Networking > All About Sockets)
Learning Network Programming with Java , R. M. Reeese, 2015

```
public class Server {
   public static void main(String args[])
   {  ServerSocket serverSocket = new ServerSocket(9090);
      System.out.println("Server is running.");            //mesaj afisat pe propriul canal
      Socket clientSocket = serverSocket.accept();
      System.out.println("Client connected!");           //mesaj afisat pe propriul canal

      BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

       String message = in.readLine();                          // mesaj primit de la client
       System.out.println("From client: " + message);  //mesaj afisat pe propriul canal

       out.println("Message received!");            // mesaj trimis clientului

      clientSocket.close();
      serverSocket.close();
}
```

```
public class Client {
   public static void main(String args[]) throws IOException
   { Socket socket = new Socket("localhost", 9090);

     PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
     BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

      Scanner clientin = new Scanner(System.in);
      String message = clientin.nextLine(); //citeste un mesaj
      out.println(message);    // mesajul este  trimis serverului

      String response = in.readLine();   // mesaj primit de la server
      System.out.println("From server: " + response);   // mesaj afisat pe propriul canal

      socket.close();
}
```

```
>Java Server
Sever is running.
Client connected!
From client: buna!
```

```
>java Client
buna!
From server: Message received!
```

```
public class Client {
   public static void main(String args[]) throws IOException
   { Socket socket = new Socket("localhost", 9090);

     PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
     BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

      Scanner clientin = new Scanner(System.in);
      String message = clientin.nextLine();
      out.println(message);    // mesaj trimis serverului

      String response = in.readLine();   // mesaj primit de la server
      System.out.println("From server: " + response);

      socket.close();
}
```

```
>Java Server
Sever is running.
Client connected!
From client: buna!
>
```

```
>java Client
buna!
From server: Message received!
>java Client
Exception ....
```

dupa ce trimite "Message received!"
serverul isi termina executia

https://docs.oracle.com/javase/tutorial/essential/concurrency

```
public class Server {
   public static void main(String args[]) throws IOException
   {  ServerSocket serverSocket = new ServerSocket(9090);
      System.out.println("Server is running");
   while (true){
      Socket clientSocket = serverSocket.accept();
     System.out.println("Client connected!");

      BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

       String message = in.readLine();
       System.out.println("From client: " + message);
       out.println("Message received!");

       clientSocket.close();
}}
```

```
>Java Server
Sever is running
Client connected!
From client: buna!
Client connected!
From client: buna din nou!
|
```

```
>java Client
buna!
From server: Message received!
>java Client
buna din nou!
From server: Message received!
>
```

```
public class Client {
   public static void main(String args[]) throws IOException
   { Socket socket = new Socket("localhost", 9090);

     PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
     BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

      Scanner clientin = new Scanner(System.in);
      String message = clientin.nextLine();

     while (!message.equals("bye")) {
      // Receive response from the server
      String response = in.readLine();
      System.out.println("From server: " + response);
      message = clientin.nextLine();
      out.println(message);
   }
      socket.close();
}
```

```
>java Client
buna!
From server: Message received!
buna!
From server: Message received!
buna!
From server: Message received!
bye
>
```

```
public class Server {
   public static void main(String args[]) throws IOException
   {  ServerSocket serverSocket = new ServerSocket(9090);
      System.out.println("Server is running");
   while (true){
      Socket clientSocket = serverSocket.accept();
     System.out.println("Client connected!");
      BufferedReader in = new BufferedReader(new InputStreamReader(clientSo
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

       String message = in.readLine();
       System.out.println("From client: " + message);
       while (! message.equals("bye")) {
              out.println("Message received!");
               message = in.readLine();
              System.out.println("From client: " + message);
            }
    clientSocket.close();
}}
```

```
>Java Server
Sever is running.
Client connected!
From client: buna!
From client: buna!
From client: buna!
From client: bye
|
```

```
>java Client
buna!
From server: Message received!
buna!
From server: Message received!
buna!
From server: Message received!
bye
>
```

https://docs.oracle.com/javase/tutorial/essential/concurrency

```
public class Server {
   public static void main(String args[]) throws IOException
   {  ServerSocket serverSocket = new ServerSocket(9090);
      System.out.println("Server is running");
   while (true){
      Socket clientSocket = serverSocket.accept();
      System.out.println("Client connected!");
      BufferedReader in = new BufferedReader(new In                    InputStream()));
      PrintWriter out = new PrintWriter(clientSocket.g

       String message = in.readLine();
       System.out.println("From client: " + message);
       while (! message.equals("bye")) {
              out.println("Message received!");
               message = in.readLine();
              System.out.println("From client: " + message);
            }
    clientSocket.close();
}}
```

```
>java Client
buna!
From server: Message received!
buna!
From server: Message received!
bye
>
```

```
>java Client
hi!
From server: Message received!
```

```
>Java Server
Sever is running.
Client connected!
From client: buna!
From client: buna!
From client: bye
Client connected!
From client: hi!
|
```

Clientii sunt serviti secvential!

https://docs.oracle.com/javase/tutorial/essential/concurrency

```
public class ServerMT implements Runnable {
private Socket clientSocket;

public static void main(String args[])
   {  ServerSocket serverSocket = new ServerSocket(9090);
      System.out.println("Server is running");
      while (true){
      Socket cSocket = serverSocket.accept();
      System.out.println("Client connected!");
      Thread clientThread = new Thread (new ServerMT (cSocket));
      clientThread.start(); }}

 public ServerMT(Socket s){this.clientSocket =s;}

 public void run()  {
      BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
       String message = in.readLine();  System.out.println("From client: " + message);
       while (! message.equals("bye")) {
        out.println("Message received!");  message = in.readLine(); System.out.println("From client: " + message);
         }
      clientSocket.close();}}
```

main si **run** trebuie scrise cu try-catch

am creat **cate un thread** pentru fiecare client

```
public class ServerMT implements Runnable {
private Socket clientSocket;

public static void main(String args[])
   {  ServerSocket serverSocket = new ServerSocket(9090);
      System.out.println("Server is running");
      while (true){
      Socket cSocket = serverSocket.accept();
      System.out.println("Client connected!");
      Thread clientThread = new Thread (new ServerMT (cSocket));
      clientThread.start(); }}

 public ServerMT(Socket s){this.clientSocket =s;}

  public void run()  {
      BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream(
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
       String message = in.readLine();  System.out.println("From client: " + message);
       while (! message.equals("bye")) {
out.println("Message received!");  message = in.readLine(); System.out.println("From client: " + mes
 }
      clientSocket.close();}}
```

```
>java Client
hi!
From server: Message received!
hi!
From server: Message received!
bye
>
```

```
>java Client
buna!
From server: Message
buna!
From server: Message received!
buna!
From server: Message received!
bye
>
```

```
va Server
er is running.
Client connected!
From client: buna!
From client: buna!
Client connected!
From client: hi!
From client: buna!
From client: hi!
From client: bye
From client: bye
```

Clientii sunt serviti **concurent**!

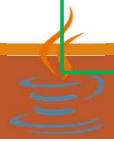
https://docs.oracle.com/javase/tutorial/essential/concurrency

Server multithreaded

```
public class ServerMT implements Runnable {
private Socket clientSocket;

public static void main(String args[])
   {  ServerSocket serverSocket = new ServerSocket(9090);

      System.out.println("Server is running");
      ExecutorService  pool = Executors.newCachedThreadPool();

      while (true){
      Socket cSocket = serverSocket.accept();
      System.out.println("Client connected!");
      pool.execute(new ServerMT (cSocket)); }}

 public ServerMT(Socket s){this.clientSocket =s;}

 public void run()  {
      BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
       String message = in.readLine();  System.out.println("From client: " + message);
       while (! message.equals("bye")) {
        out.println("Message received!");  message = in.readLine(); System.out.println("From client: " + message);}
       clientSocket.close();}}
```

main si **run** trebuie scrise cu try-catch

creez thread-urile client cu **ExecutorService**



https://docs.oracle.com/javase/tutorial/essential/concurrency

JEP 444: Virtual Threads

The thread-per-request style

Server applications generally handle concurrent user requests that are independent of each other, so it makes sense for an application to handle a request by dedicating a thread to that request for its entire duration. This thread-per-request style is easy to understand, easy to program, and easy to debug and profile because it uses the platform's unit of concurrency to represent the application's unit of concurrency. […]Unfortunately, the number of available threads is limited because the JDK implements threads as wrappers around operating system (OS) threads.

Improving scalability with the asynchronous style

Some developers wishing to utilize hardware to its fullest have given up the thread-per-request style in favor of a thread-sharing style. In the asynchronous style, each stage of a request might execute on a different thread, and every thread runs stages belonging to different requests in an interleaved fashion. […] *This programming style is at odds with the Java Platform because the application's unit of concurrency — the asynchronous pipeline — is no longer the platform's unit of concurrency.*

Preserving the thread-per-request style with virtual threads

To enable applications to scale while remaining harmonious with the platform, we should strive to preserve the thread-per-request style. We can do this by implementing threads more efficiently, so they can be more plentiful. Operating systems cannot implement OS threads more efficiently because different languages and runtimes use the thread stack in different ways. It is possible, however, for a Java runtime to implement Java threads in a way that severs their one-to-one correspondence to OS threads. Just as operating systems give the illusion of plentiful memory by mapping a large virtual address space to a limited amount of physical RAM, a Java runtime can give the illusion of plentiful threads by mapping a large number of virtual threads to a small number of OS threads.

➢ Server multithreaded cu thread-uri virtuale

```
public class ServerMTv implements Runnable {
private Socket clientSocket;

public static void main(String args[])
   {  ServerSocket serverSocket = new ServerSocket(9090);
      System.out.println("Server is running");
      ExecutorService  pool = Executors.newVirtualThreadPerTaskExecutor();
      while (true){
      Socket cSocket = serverSocket.accept();
      System.out.println("Client connected!");
      pool.execute(new ServerMT (cSocket)); }}

 public ServerMTv(Socket s){this.clientSocket =s;}

  public void run()  {
      BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
      PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
       String message = in.readLine();  System.out.println("From client: " + message);
       while (! message.equals("bye")) {
        out.println("Message received!");  message = in.readLine(); System.out.println("From client: " + message);}
       clientSocket.close();}}
```

am creat o piscina **de thread-uri virtuale**

JEP 444: Virtual Threads

The thread-per-request style […]

Improving scalability with the asynchronous style

Some developers wishing to utilize hardware to its fullest have given up the thread-per-request style in favor of a thread-sharing style. In the asynchronous style, each stage of a request might execute on a different thread, and every thread runs stages belonging to different requests in an interleaved fashion. […] *This programming style is at odds with the Java Platform because the application's unit of concurrency — the asynchronous pipeline — is no longer the platform's unit of concurrency.*

Preserving the thread-per-request style with virtual threads […]

➢ NIO (New Input/Output)

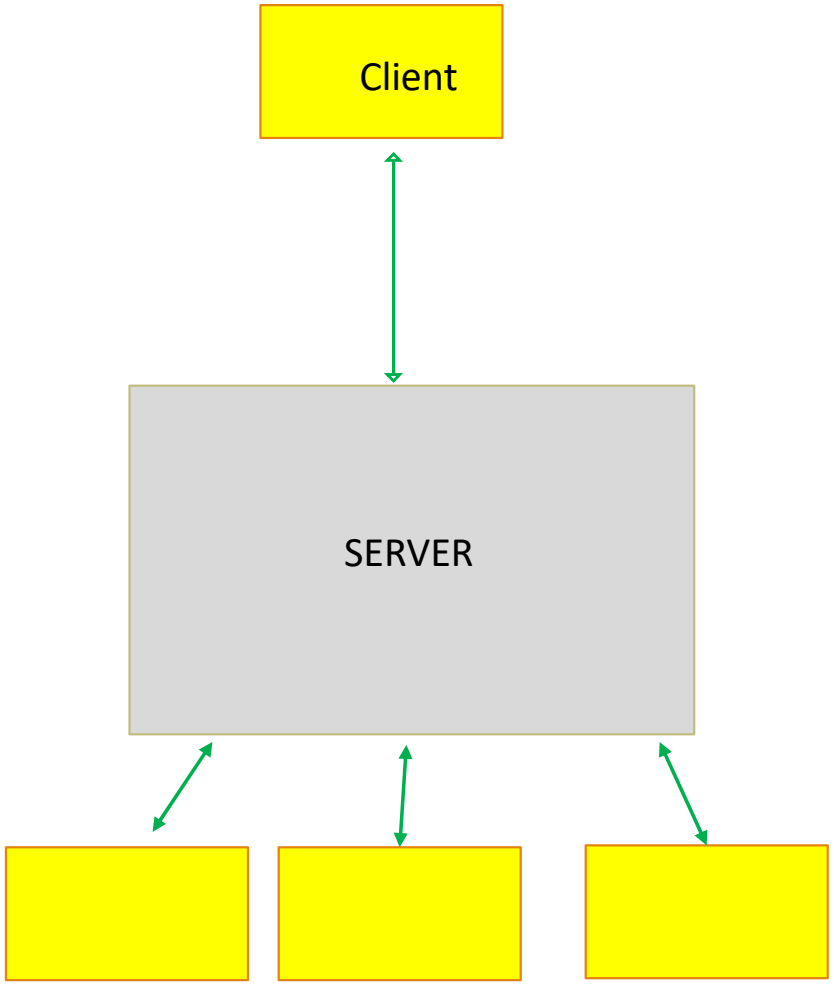
- API-ul NIO foloseste canale si buffere (spre deosebire de API-ul IO care foloseste stream-uri).

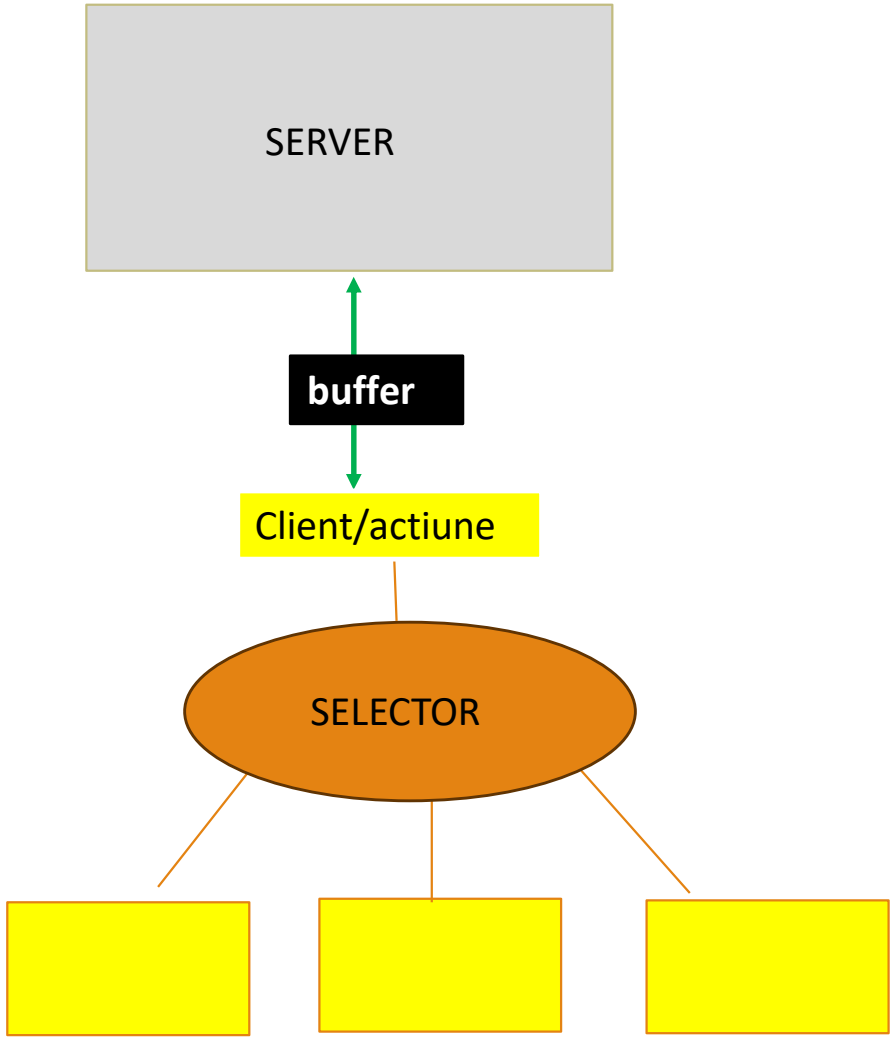
- **Bufferele** sunt blocuri de memorie in care datele sunt scrise pentru a fi prelucrate ulterior. Operatiile de citire/scriere a bufferelor se fac fara blocarea thread-urilor (spre deosebire de operatiile de citire/scriere a stream-urilor).

- **Canale** sunt conexiuni intre buffere si entitati care efectueaza operatiile de intrare/iesire (fisiere sau socket-uri): datele sunt scrise de pe canal intr-un buffer sau citite dintr-un buffer pe un canal. **SocketChannel** sunt conectate la un socket iar **ServerSocketChannel** poate accepta conexiuni (TCP)

- **Selectori** care realizeaza multiplexarea operatiilor de intrare-iesire de pe canale selectabile. Astfel, un sigur thread poate servi mai multe canale. Pentru a fi procesate cu ajutorul unui selector canelele sunt inregistrate de selectorul respectiv, iar metoda **select()** a selectorului blocheaza thread-ul pana cand cel putin un canal inregistrat este gata pentru prelucrare.

[java.nio (Java SE 24 & JDK 24)]


Client
SERVER


SERVER
buffer
Client/actiune
SELECTOR

➢ **Server NIO event-driven (un singur thread)**

```
public class NioServer {
  public static void main(String[] args) {
    try{
    Selector selector = Selector.open();
    ServerSocketChannel serverSocket =   ServerSocketChannel.open();
    serverSocket.bind(new InetSocketAddress("localhost", 9090));
    serverSocket.configureBlocking(false);
    serverSocket.register(selector, SelectionKey.OP_ACCEPT);
    System.out.println("Server looking for clients ...");
    ByteBuffer buffer = ByteBuffer.allocate(1024);
    while (true) {
      selector.select();
      Set<SelectionKey> selectedKeys = selector.selectedKeys();
      for (SelectionKey key : selectedKeys) {
                                  …
                              } }
     } catch (IOException e) {
        throw new RuntimeException(e);
     }   }}
```

thread-ul nu este blocat de operatiile I/O sau conexiuni pe canalul respectiv

```
public class NioServer {
  public static void main(String[] args) {
    try{
    Selector selector = Selector.open();
    ServerSocketChannel serverSocket =   ServerSocketChannel.open(
    serverSocket.bind(new InetSocketAddress("localhost", 9090));
    serverSocket.configureBlocking(false);
    serverSocket.register(selector, SelectionKey.OP_ACCEPT);
    System.out.println("Server looking for clients ...");
    ByteBuffer buffer = ByteBuffer.allocate(1024);
    while (true) {
      selector.select();
      Set<SelectionKey> selectedKeys = selector.selectedKeys();
      for (SelectionKey key : selectedKeys) {
                                ...
                              } }
     } catch (IOException e) {
        throw new RuntimeException(e);
     }   }}
```

➢ **Server NIO event-driven (un singur thread)**

```
if (key.isAcceptable()) {
          SocketChannel client = serverSocket.accept();
          client.configureBlocking(false);
          client.register(selector, SelectionKey.OP_READ);
        }
        else if (key.isReadable()) {
          SocketChannel client = (SocketChannel) key.channel();
          buffer.clear();
          var bytesRead = client.read(buffer);
          if (bytesRead == -1) { client.close();
            System.out.println("Client disconnected!");}
          else {String result = new String(buffer.array()).trim();
          System.out.println("Message received: " + result);}
              }
        else {
          throw new RuntimeException("Unknown channel");
       }
        selectedKeys.remove(key);
```

- **Client - Server NIO event-driven (un singur thread)**

```
public class NioClient{
   public static void main(String[] args) {
      var scanner = new Scanner(System.in);
      try (var serverChannel = SocketChannel.open()) {
         serverChannel.connect(new InetSocketAddress(9090));
         serverChannel.configureBlocking(false);
         System.out.println("Connection established! Stop with \"quit\" ");
        var buffer = ByteBuffer.allocate(1024);
         while (true) {
            var line = scanner.nextLine();
            if (line.equalsIgnoreCase("quit")) {
               break;
            }
             buffer.clear().put(line.getBytes()).flip();
            serverChannel.write(buffer);
}
      } catch (IOException e) {
         throw new RuntimeException(e);
      }
   }
}
```

- **clear()** makes a buffer ready for a new sequence of channel-read or relative put operations:
  it sets the limit to the capacity and the position to zero.

- **flip()** makes a buffer ready for a new sequence of channel-write or relative get operations: it sets the limit to the current position and then sets the position to zero.

➢ **Client - Server NIO event-driven (un singur thread)**

```
public class NioServer {
  public static void main(String[] args) {
    try{
    Selector selector = Selector.open();
    ServerSocketChannel serverSocket =   ServerSocketChanne
    serverSocket.bind(new InetSocketAddress("localhost", 909
    serverSocket.configureBlocking(false);
    serverSocket.register(selector, SelectionKey.OP_ACCEPT);
    System.out.println("Server looking for clients ...");
    ByteBuffer buffer = ByteBuffer.allocate(1024);
    while (true) {
      selector.select();
      Set<SelectionKey> selectedKeys = selector.selectedKeys()
      for (SelectionKey key : selectedKeys) {
                                …
                              } }
     } catch (IOException e) {
        throw new RuntimeException(e);
     }  }}
```

```
java NioClient
Connection established! Stop
with "quit"
c2m1
c2m2
c2m3
quit
```

```
java NioClient
Connection established! Stop
with "quit"
c1m1
c1m2
quit
```

```
java NioServer
Server looking for clients ...
Message received: c1m1
Message received: c2m1
Message received: c2m2
Message received: c2m3
Message received: c1m2
Client disconnected!
Client disconnected!
|
```

➢ **Client-Server cu canale asincrone NIO2**

```
public class Nio2Server{
  public static void main(String[] args) throws Exception {
    AsynchronousServerSocketChannel server  = AsynchronousServerSocketChannel.open();
    server.bind(new InetSocketAddress("localhost", 9090));
    Future<AsynchronousSocketChannel> acceptFuture = server.accept();
      AsynchronousSocketChannel clientChannel = acceptFuture.get();
      while ((clientChannel != null) && (clientChannel.isOpen())) {
        ByteBuffer buffer = ByteBuffer.allocate(32);
        Future<Integer> readResult  = clientChannel.read(buffer);

        readResult.get();

        buffer.flip();
        Future<Integer> writeResult = clientChannel.write(buffer);

        writeResult.get();
        buffer.clear();
      }
      clientChannel.close(); server.close();}}
```

canale care permit operatii de intrare/iesire asincrone.
AsynchronousChannel (Java SE 24 & JDK 24)

interactiune cu un singur client

➢ **Client - Server cu canale asincrone NIO2**

```
public class Nio2Client {
   public static void main(String[] args) throws Exception {

AsynchronousSocketChannel client = AsynchronousSocketChannel.open();
InetSocketAddress hostAddress = new InetSocketAddress("localhost", 9090);
Future<Void> future = client.connect(hostAddress);    // poate executa alte actiuni
future.get();
System.out.println("Connection established! Stop with \"quit\" ");
var scanner = new Scanner(System.in);
while (true){
var line = scanner.nextLine();
if (line.equalsIgnoreCase("quit")) { break;}
   byte[] byteMsg = new String(line).getBytes();
   ByteBuffer buffer = ByteBuffer.wrap(byteMsg);
   Future<Integer>  writeResult = client.write(buffer);     // poate executa alte actiuni
   writeResult.get();
   buffer.flip();
   Future<Integer>  readResult = client.read(buffer);   // poate executa alte actiuni
   readResult.get();
   String message = new String(buffer.array()).trim();  buffer.clear();System.out.println(message);}
   client.close();}}
```

➢ **Client-Server canale asincrone NIO2**

```
public class Nio2Server{
  public static void main(String[] args) throws Exception {
    AsynchronousServerSocketChannel server  = AsynchronousServerSocketChannel.open();
    server.bind(new InetSocketAddress("localhost", 9090));
    ExecutorService  pool = Executors.newCachedThreadPool();
    while (true) {
      Future<AsynchronousSocketChannel> acceptFuture = server.accept();
      AsynchronousSocketChannel clientChannel = acceptFuture.get();
      pool.execute( new Runnable(){public void run() {
        try{
      while ((clientChannel != null) && (clientChannel.isOpen())) {
        ByteBuffer buffer = ByteBuffer.allocate(32);
        Future<Integer> readResult  = clientChannel.read(buffer);
        readResult.get();
        buffer.flip();
        Future<Integer> writeResult = clientChannel.write(buffer);

        writeResult.get();
        buffer.clear();
      }
       clientChannel.close();}
        catch (Exception ex) {ex.printStac
```

Asynchronous channels are safe for use by multiple concurrent threads. Some channel implementations may support concurrent reading and writing
AsynchronousChannel (Java SE 24 & JDK 24)